Why Accurate Models Are Easy to Fool: A Hands-On Tour of Adversarial Vulnerability

Ali Kayyam
akayyam@brainchip.com

Abstract

This paper is an illustrative tutorial for readers without specialist background, students and practitioners included, built around one question: why can tiny changes fool an accurate image classifier? We build the smallest problem where the answer can be checked exactly, a task with one reliable clue and five weak clues, train a small neural network on it, attack it, and take the problem apart one question at a time, each with a small experiment and a figure. The answer that emerges is a candidate for the main cause: the model's answer is a sum of many small pieces of evidence, and an attacker who may change every input a little can push all of them the same way. Spreading the same evidence over more inputs leaves accuracy unchanged but shrinks the change needed per input by the square root of the number of inputs (eight times smaller at 64 inputs), while the *total* size of the change needed does not move. No single weak clue is to blame; their sum is. A second lesson is that how *reliable* a clue is does not make it safe. What matters is how *small* the clue is compared with the allowed change, so a clue that is 99.9% reliable can still be flipped by a tiny change. We then show which fixes work and what they cost, when the effect appears, and how well the picture carries over to real images (circles and squares, MNIST, CIFAR-10), and we discuss what it does and does not say about why networks are vulnerable, including links to shape versus texture, one-pixel attacks and geometric distortions. It is an aid to intuition, not a claim about large deep networks in general.

1 The idea in plain words

A photo of a panda, plus a faint pattern of noise a person cannot see, is labelled "gibbon", with high confidence, by a network that read the original correctly as "panda" (a famous example from Goodfellow et al. [2]). This is an *adversarial example*. Figure 1 shows a real one from a small digit classifier: a handwritten digit it reads correctly with full confidence is turned into a different digit by changing every pixel by at most 0.05 or 0.06 (on a scale where pixels span 1), a change a person would barely notice.

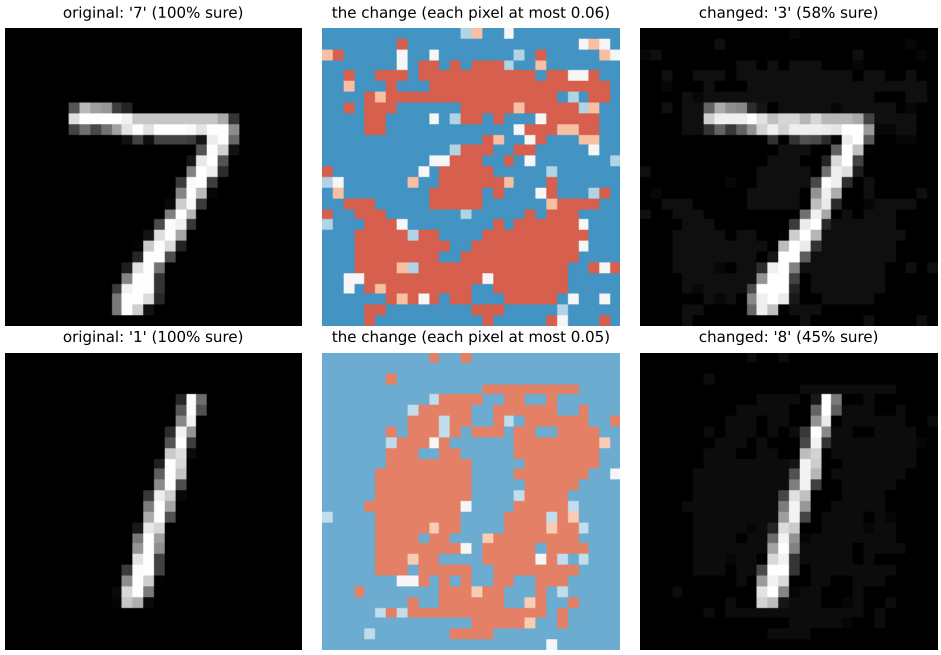


Figure 1: Two real adversarial examples from a small digit network. Left: the original digit and the network's answer. Middle: the change (red: brightened, blue: darkened; no pixel moves by more than the amount shown). Right: the changed digit and the network's new answer.

Two explanations are often given:

- **Non-robust features** (Ilyas et al. [9]). Images contain clues that really do predict the label, but are fragile. A network trained to be accurate uses them, so it inherits their fragility.
- **Linearity in high dimension** (Goodfellow et al. [2]). A network reads thousands of numbers. A tiny nudge to each of them, all in the unhelpful direction, adds up to a big change in the answer.

These can sound like rivals. This paper uses a single toy problem as a bridge between them: in it, a clue can be perfectly good evidence and still make the model fragile, simply because the evidence is spread over many small contributions that an attacker can push together. The experiments below show which parts of each idea hold up, and where.

The question. The question this paper tries to answer is: *what is the main cause of adversarial vulnerability?* Our candidate answer, tested in a toy world and then checked against real images, is that an accurate model's answer is a sum of many small contributions, and a small change to every input moves all of them together. Section 11 says how far that answer carries and where it does not.

In plain words. Think of a vote among many voters. If the voters are independent, nudging one of them does little. But an attacker who can whisper a tiny bias to *every* voter at once can swing the vote, even though no single voter was ever strongly persuaded.

How to read this paper. Each section poses a question, runs a small experiment, and gives the answer in a box. Appendix B is a short glossary. Numbers are averages over 10 random training runs (mean ± one standard deviation) unless stated; Appendix A gives the exact settings, and all code is at `https://github.com/alikayyam/robust`.

Where this sits. Szegedy et al. [1] first described adversarial examples. Goodfellow et al. [2] introduced the one-step attack (FGSM) used here, and Madry et al. [3] its iterated version (PGD). Ilyas et al. [9] proposed the robust/non-robust decomposition we test. Tsipras et al. [5] argued that robustness and accuracy can be at odds, which our trade-offs reproduce in miniature. Athalye et al. [4] showed that many defenses fail once the attacker adapts; that is why we measure here and do not propose a defense.

2 A toy world

Every example is six numbers $x_0, \ldots, x_5$ and a label that is 0 or 1 with equal chance (like a fair coin flip). Each number is an ordinary real value centred on zero: typically negative for label 0 and positive for label 1. The reliable clue typically lies between about -1.4 and 1.4 (average ∓ 0.5, wobble 0.3), and a weak clue between about -1 and 1 (average ∓ 0.2, wobble 0.25).

- x_0 is the **reliable clue**. Its average value is higher for label 1 than for label 0, with a moderate amount of random wobble. Looking at x_0 alone and picking the nearer class gets the label right 95% of the time.
- $x_1, \ldots, x_5$ are the **weak clues**. Each also shifts with the label, but the shift is small compared with the wobble, so any one of them alone gets the label right only 79% of the time.

Everything is known by construction, so we can compute, for any model, exactly how good it could possibly be. The best possible classifier (called *Bayes-optimal*; we also call it "the best rule") gets 99.3%. The wobble is bell-curve noise and the numbers are not clipped to a range, which is what makes the best classifier a plain weighted sum of the six clues.

How the decision is made. The best rule multiplies each clue by a weight (larger for clues that separate the classes better), adds the results into one total, and answers label 1 if the total is above zero and label 0 otherwise. The size of the total is the rule's *confidence*: far from zero means a confident answer, near zero a close call. Table 1 applies the same rule to the listed clues only. A trained network is not told this rule; it has to learn a rule from examples.

Table 1: How good is each clue, and how good are they together? Accuracy of the best possible rule on each set of clues (mean ± s.d. over 10 test sets).

Clues used	Best possible accuracy
reliable clue x_0 alone	0.955 ± 0.006
one weak clue alone	0.789 ± 0.009
all five weak clues together	0.963 ± 0.009
all six clues	0.993 ± 0.002

In plain words. Five clues that are each only so-so are, together, *better* than the one good clue. "Weak" does not mean "useless." It means weak one at a time.

Figure 2 shows why. One weak clue alone (middle) barely separates the two classes. But the *average* of the five weak clues (right) separates them as well as the reliable clue does, because the wobble in independent clues partly cancels when they are averaged.

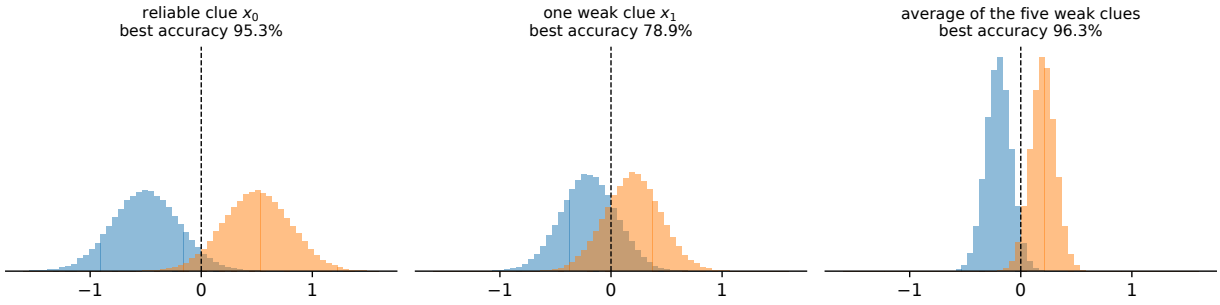


Figure 2: The two classes (blue: label 0, orange: label 1) as seen through the reliable clue, one weak clue, and the average of the five weak clues. The dashed line is the best single threshold; the title gives the resulting accuracy.

Three models. We train a tiny neural network (one hidden layer of 8 units) in three ways (Figure 3). **A** sees all six numbers. **B** sees only the reliable clue; the weak clues have no input at all. **C** has input slots for all six but the weak ones are always set to zero.

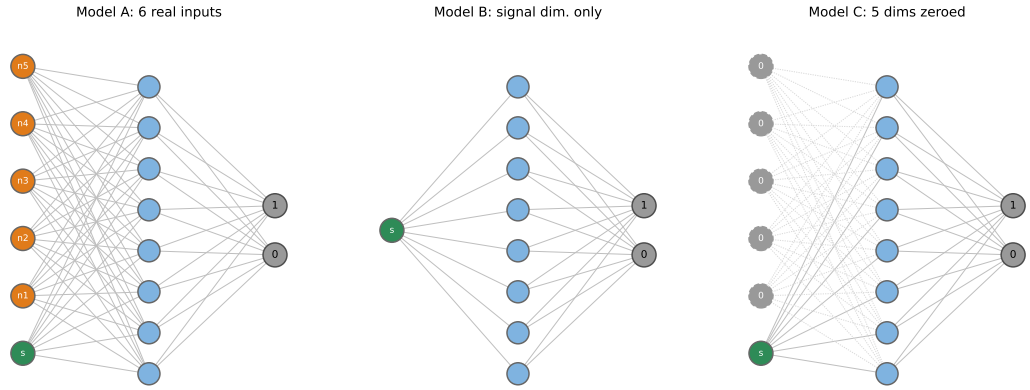


Figure 3: The three models. Green: the reliable clue. Orange: the weak clues (only A sees their values). Gray: slots that are always zero (C only).

3 How we attack

The attacker may change each input number by at most a small amount ϵ (we try ϵ up to 0.3). This is the standard "per-input" (ℓ_∞) budget; the alternative, a limit on the *total* (straight-line, ℓ_2) size of the change, will matter in Section 4. The attacker chooses the direction of each change to hurt the model most, using the model's own gradient (this is the standard one-step FGSM attack). We count the share of test points that the model got right before the attack and gets wrong after it, so a "flip" always means correct → incorrect. We can also forbid the attacker from touching some inputs, which lets us ask *where* the model is vulnerable.

A nudge has to be judged against the *size* of the clue it is applied to. Here the reliable clue's two class averages are 1.0 apart (half-gap 0.5) and each weak clue's are 0.4 apart (half-gap 0.2), so a nudge of 0.3 is already larger than a weak clue's half-gap. A very large nudge would make anything look fragile; the sizes we use are chosen to be informative, not extreme.

In plain words. The attack is not random noise. It is the worst possible nudge of a given size for the specific model.

Is a one-step attack enough? We also ran a stronger, iterated attack (PGD: 20 small steps from random starting points, 4 restarts) against every model and subset of inputs. On the toy task it flips *exactly* the same points as the one-step attack, at every nudge size and in every one of the 10 runs (the largest per-run difference is 0.000); for example 0.536 against 0.536 for all six inputs at $\epsilon = 0.3$. The reason is that for this tiny network the direction that hurts most does not change between the clean point and the edge of the allowed box, so there is nothing for extra steps to improve. This is a property of this small network and task, not a general fact. On the image network of Section 10 the iterated attack is stronger than the one-step attack.

4 Question 1: what happens when the same evidence is spread over more inputs?

Recall that the best rule multiplies each input x_i by a weight w_i, adds the results into a total $\sum_i w_i x_i$, and answers by the sign of the total; the size of the total is its confidence (Section 2). Now an attacker nudges every input by at most ϵ, each in the direction that lowers the total. Input i then moves the total by $\epsilon|w_i|$, so all inputs together move it by $\epsilon \sum_i |w_i|$. The answer flips as soon as that shift exceeds the confidence. So the smallest nudge per input that flips an answer is

$$\text{flip size} = \frac{\text{confidence of the answer}}{\sum_i |w_i|}.$$

Each input contributes its own $|w_i|$ to the denominator, so *more inputs carrying the evidence means a smaller flip size*. The sum of absolute weights, $\|w\|_1$, is the whole story for a weighted sum (Figure 4). In our task the reliable clue has weight 11.1 and each weak clue 6.4, so the five weak clues together carry 32.0, nearly three times as much. *Illustration (made-up confidence of* 5*):* if the evidence sits in the reliable clue alone, the flip size is $5/11.1 \approx 0.45$. If the answer rests on the five weak clues, it is $5/32.0 \approx 0.16$: the same confidence is overturned by a per-input nudge about three times smaller, because five inputs are being pushed at once.

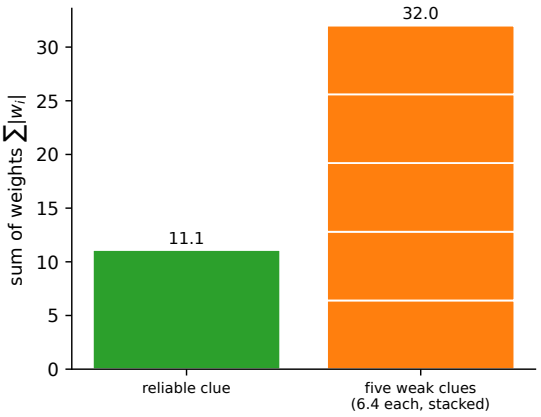


Figure 4: The sum of weights $\sum |w_i|$, which sets how far a full nudge moves the answer. One reliable clue carries 11.1; the five weak clues, stacked, carry 32.0.

A picture helps (Figure 5). Draw the allowed changes for two inputs as a square (each input may move by at most ϵ) and the decision boundary as a line. If all the evidence is in one input, the boundary is a vertical line and the square must grow until its *side* reaches it. If the same evidence is split over two inputs, the boundary is diagonal, and the square's *corner*, which moves both inputs at once, reaches it sooner. The straight-line distance to the boundary (blue circle) is the same in both cases; only the square, the per-input change, shrinks, by $1/\sqrt{2}$ here and by $1/\sqrt{k}$ for k inputs.

In symbols: if k inputs carry equal weights $c/\sqrt{k}$, the straight-line size of the weights is $\|w\|_2 = c$ whatever k is, while the sum of absolute weights is $\|w\|_1 = c\sqrt{k}$. A margin M then needs a total change of $M/\|w\|_2 = M/c$ but only a per-input change of $M/\|w\|_1 = M/(c\sqrt{k})$. So the per-input flip size is the total flip size divided by $\sqrt{k}$.

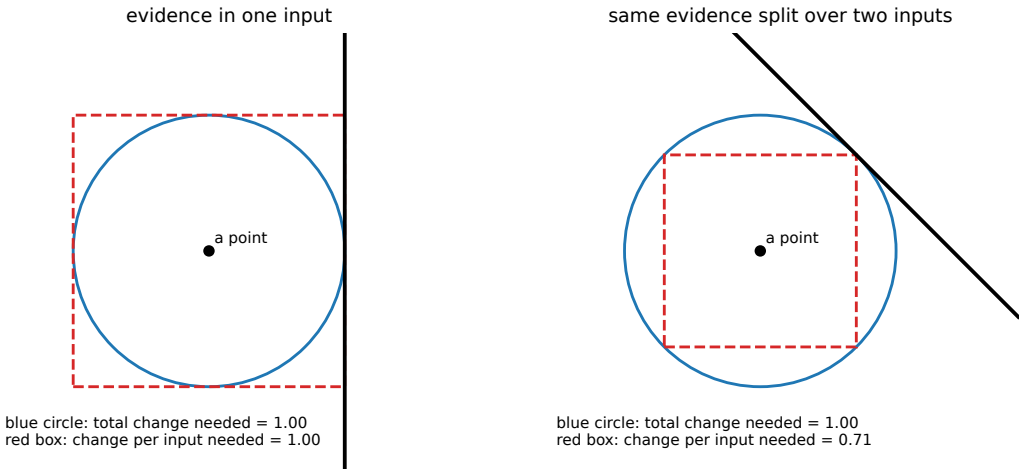


Figure 5: Two inputs, one point. Blue circle: total change needed to reach the decision boundary (black). Red dashed square: per-input change needed. Splitting the evidence over two inputs leaves the circle unchanged and shrinks the square.

4.1 The same evidence, spread out

To test this directly (Figure 6) we keep the total amount of evidence fixed and split it equally over k inputs, $k = 1, 2, 4, \ldots, 64$. A classifier with the same accuracy (95%) is built each time.

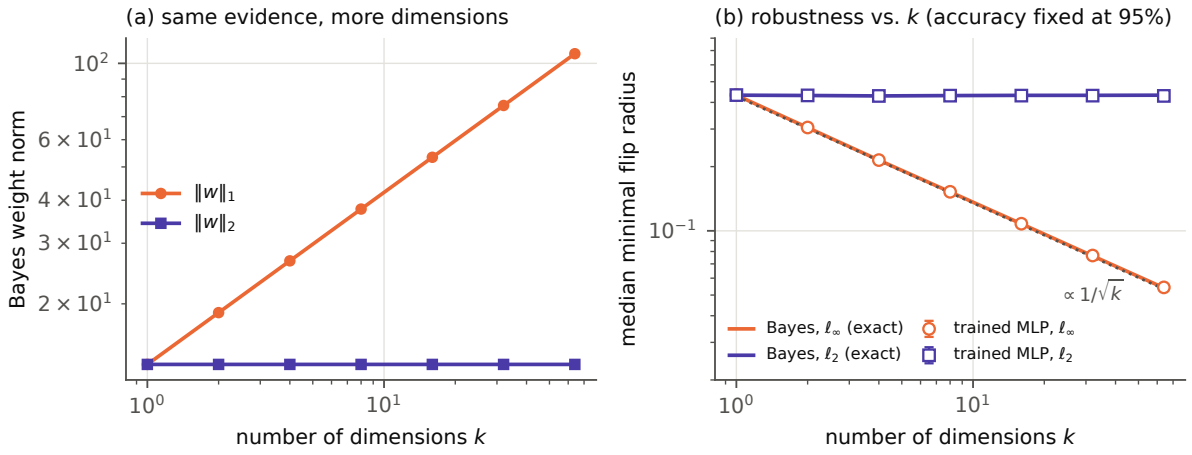


Figure 6: Spreading a fixed amount of evidence over more inputs. Right panel: the smallest per-input change that flips an answer (median) falls as $1/\sqrt{k}$ (steep lines) while the *total size* of the change needed (the ℓ_2 distance, flat lines) stays put. Exact best rule (lines) and trained networks (markers) coincide.

With $k = 1$ the per-input flip size is 0.43; with $k = 64$ it is 0.054, eight times smaller, at the same accuracy. There are two ways to measure how big an attack is. One is the largest change made to any single input (the per-input, or ℓ_∞, size). The other is the length of the whole change taken together, like the length of a path (the total, or ℓ_2, size). For example, nudging 64 inputs by 0.05 each has per-input size 0.05 but total size $0.05\sqrt{64} = 0.4$. Measured by total size, the change needed to flip an answer is 0.43 at every k: in that sense the model is no easier to fool. The trained network follows the exact curves of the best rule at every k.

In plain words. Splitting the same information over more inputs makes the model eight times easier to fool when the attacker is limited by how much *each input* may change (ℓ_∞), without making it less accurate and without making any individual input more fragile. If the attacker is instead limited by the *total* size of the change (ℓ_2), nothing changes. Which budget matters for a real attack is a modelling choice; ℓ_∞ is the common one for images because it mirrors "every pixel changes only slightly".

5 Question 2: who does the network listen to?

The network (A) reaches 99.4% accuracy, matching the best possible rule of Section 2 (99.3%). But does it use the weak clues the way the best rule would? To compare fairly, we *scramble* one group of clues: we shuffle its values among the test examples, so each example receives another example's value for that clue. The group then carries no information about the label, while everything else stays the same. We apply the same scrambling to the best rule and to the network and see how much accuracy each loses (Table 2). The bigger the drop, the more the rule depended on that group.

Table 2: Accuracy lost when a group of clues is scrambled (mean ± s.d. over 10 runs).

	reliable scrambled	weak scrambled	ratio (weak÷reliable)
best possible rule	0.213 ± 0.009	0.154 ± 0.014	0.72
trained network (A)	0.245 ± 0.011	0.134 ± 0.017	0.55

Figure 7 shows the same numbers as bars. The last column of the table divides the accuracy lost by scrambling the weak clues by the accuracy lost by scrambling the reliable clue: the larger the ratio, the more the rule leans on the weak clues relative to the reliable one.

In plain words. The network leans on the weak clues a bit *less*, relative to the reliable one, than the best possible rule does. So the common story, "the network latches onto weak spurious clues too much," is not what is happening here. The weak clues are genuinely predictive (just individually weak), and they are used about as much as they deserve; that is already enough to be exploited. The difference is modest, and we would not claim it holds for other networks or training recipes.

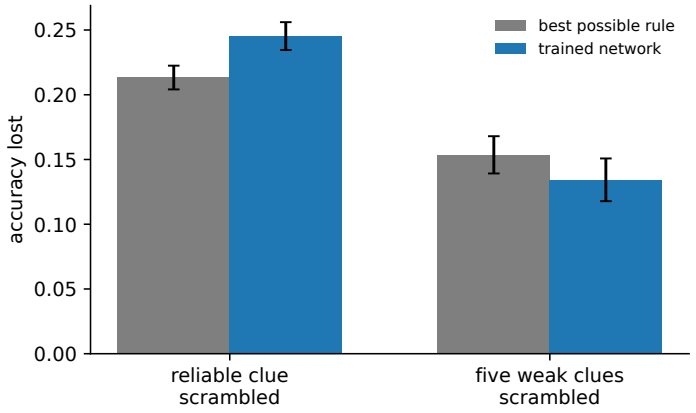


Figure 7: Accuracy lost when a group of clues is scrambled, for the best possible rule (gray) and the trained network (blue). Error bars: one standard deviation over 10 runs.

5.1 Is it the network's fault?

We can also attack the *best possible* classifier, which involves no training at all. It is just as vulnerable as the trained one: the median per-input flip size is 0.277 for the best rule and 0.285 for the network (Figure 8 compares them over all nudge sizes).

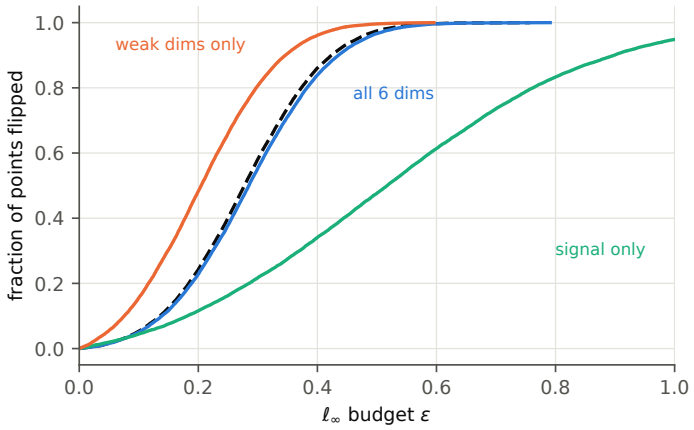


Figure 8: Share of test points flipped as the allowed nudge grows. The best possible rule (dashed black) lies on top of the trained network (blue). Networks that see only the five weak clues (orange) or only the reliable clue (green) are shown for comparison.

In plain words. The network is not doing something silly that a better-trained or better-designed network would avoid. The fragility belongs to the problem and to the kind of attack allowed.

6 Question 3: where is the network vulnerable?

We attack Model A while allowing the attacker to touch only some inputs. Table 3 gives the results at one nudge size, and Figure 9 shows them for all nudge sizes.

Table 3: Share of the correctly classified test points that the attack turns into errors (correct $\rightarrow$ incorrect), at $\epsilon = 0.3$ (mean $\pm$ s.d. over 10 runs). Clean accuracy: A 0.994, B 0.956, C 0.956.

What the attacker may change	Flipped
all six inputs (Model A)	0.536 ± 0.020
the reliable clue only	0.037 ± 0.005
all five weak clues	0.260 ± 0.014
just one weak clue	0.012 ± 0.003
Model B (reliable clue only), attacker may change it	0.211 ± 0.014

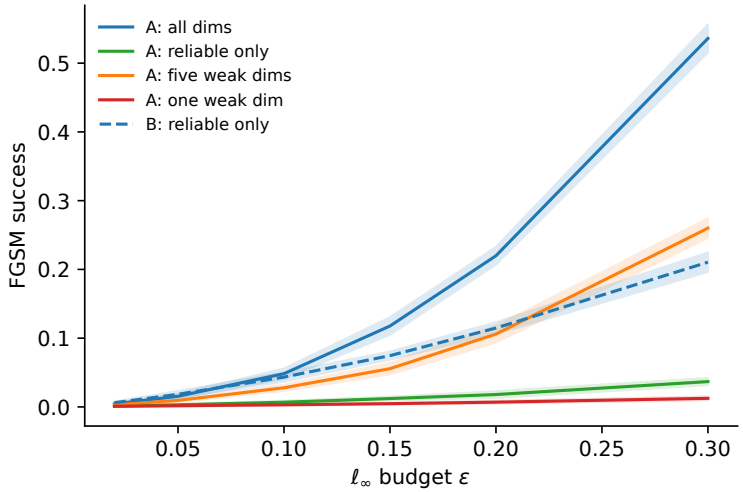


Figure 9: Share flipped against attack size ϵ. One weak clue (red) is the hardest thing to exploit; five together (orange) are far easier than the reliable clue alone (green).

The first reading of the table is tempting: attacking the five weak clues is several times as effective as attacking the reliable one (0.26 against 0.04), so *the weak clues are the weak spot.* The row "just one weak clue" shows why that is misleading. A single weak clue flips only 1.2%, *less* than the reliable one. The first comparison was five coordinates against one, not weak against strong.

Figure 10 shows the mechanism on one test point. Each weak clue, nudged by 0.3, takes a little off the network's confidence. Nudging only the reliable clue leaves the point correctly classified (Figure 10). After four weak nudges the answer flips.

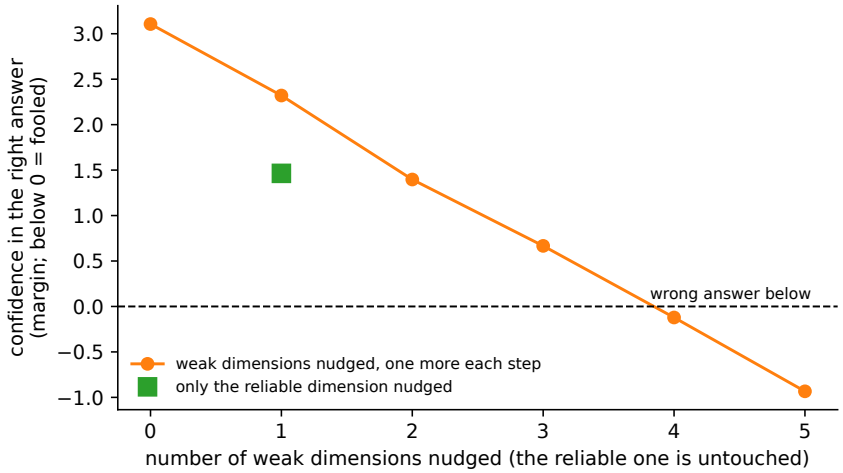


Figure 10: One test point (Model A; a typical point among those where nudging the reliable clue alone does not flip it). Orange: the model's confidence in the right answer as more weak clues are nudged by 0.3, one more at each step; the answer flips at four. Green: nudging the reliable clue alone. Below the dashed line the model is fooled.

Averaged over all test points and runs, the same story appears in Figure 11: the share flipped climbs steadily as the attacker is allowed to touch one, two, ..., five weak clues, and each added clue adds more than the one before.

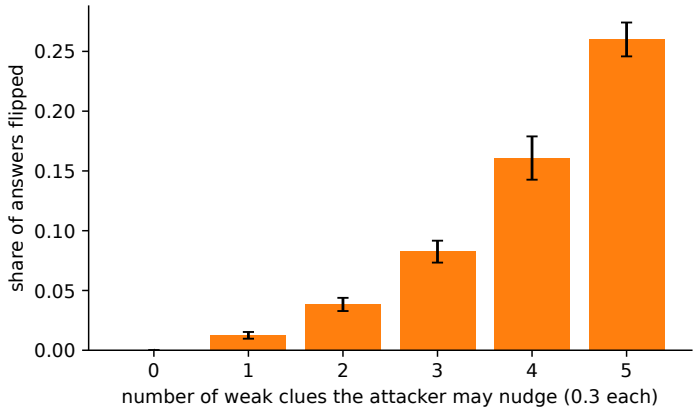


Figure 11: Share of answers flipped (Model A, nudge 0.3) as the attacker may nudge more and more of the weak clues. The reliable clue is never touched. Error bars: one standard deviation over 10 runs.

Two more views of the same fact come from a single training run each. Figure 12 slices the network's decision along each input in turn: the reliable clue gives a steep, well-separated curve, each weak clue a shallow ramp. A shallow ramp per clue is exactly why one weak nudge does little; five of them add up. Figure 13 shows one real attack: with the reliable clue untouched, the combined shift in five weak clues alone flips the answer.

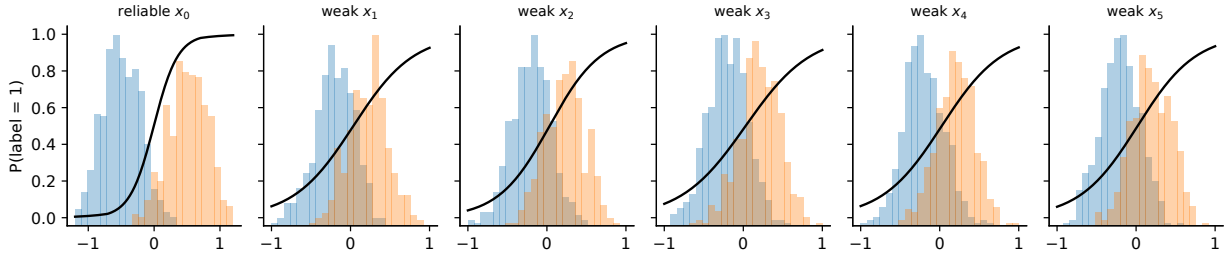


Figure 12: One training run. The network's predicted probability of label 1 (black) as each input is varied alone (others held at their average), with the two classes' distributions behind (blue: label 0, orange: label 1).

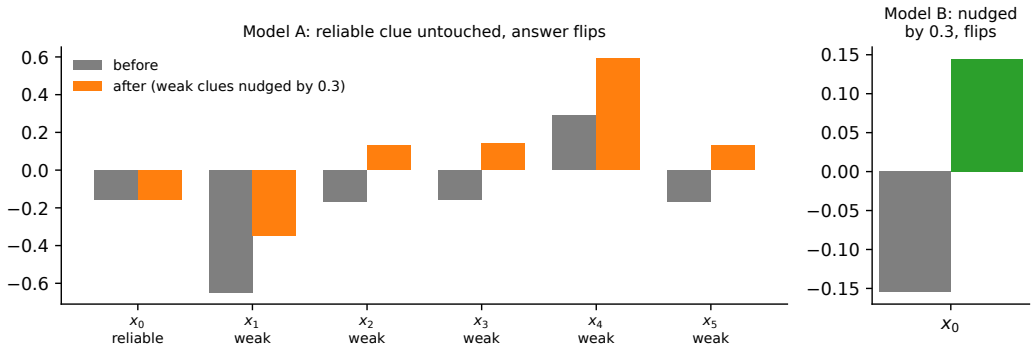


Figure 13: One successful attack per model at $\epsilon = 0.3$ on one test point, one training run. Left: attack allowed only on the weak clues; the reliable clue is unchanged yet the answer flips. Right: the model that sees only the reliable clue, attacked directly.

Figure 14 repeats the comparison in terms of *flip size*, the smallest nudge needed, which is easier to interpret than a share: attacking the reliable clue alone needs a nudge of about 1.1; attacking one weak clue needs 1.9 (harder); attacking all five weak clues needs 0.37 (much easier); attacking everything needs 0.28. The exact best rule and the trained network agree.

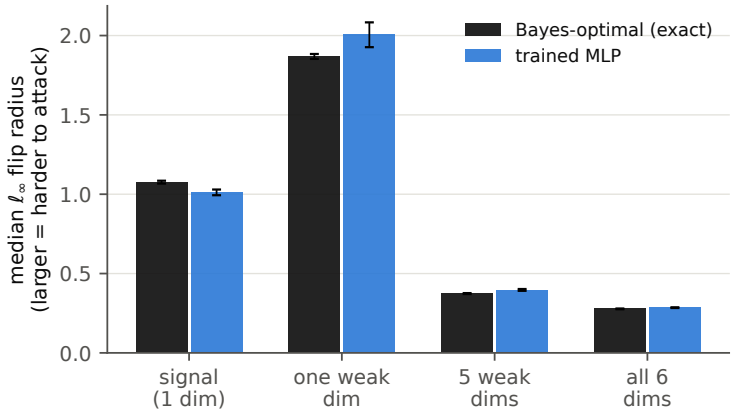


Figure 14: Median flip size (the smallest nudge that flips a point; larger means harder to attack) when the attacker controls only the listed inputs. Black: exact best rule. Blue: trained network (mean ± s.d. over 10 runs).

In plain words. No single weak clue is fragile. The weak clues are dangerous *together*, because their small effects pile up when the attacker pushes them all the same way. "Attacks through the weak features work better" is true only because there are five of them.

7 Question 4: is a reliable clue a safe clue?

It is natural to think: if a clue is 99.9% predictive of the label, surely it must be robust. It is not. Reliability (how well a clue predicts the label) and fragility (how small a nudge flips it) are different quantities, and the second depends on the clue's *size* relative to the nudge.

"Weak" has so far meant "unreliable." But the notion of "non-robust" in the original theory is about a clue being *small* relative to the nudge, not unreliable. To separate the two, we took a single clue and varied them independently: its *size* (the gap between the two class averages) and its *reliability* (how well it predicts the label). Figure 15 and Table 4 show the result.

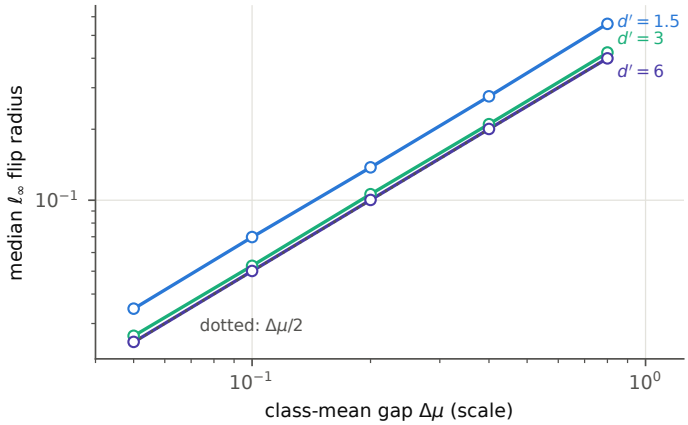


Figure 15: Flip size of a single-clue classifier against the clue's size (class gap), at three reliabilities (log scales). The lines are almost on top of one another: size matters, reliability hardly does. Lines: exact; markers: trained network.

Table 4: Smallest per-input change that flips a single-clue classifier (median). Rows: clue size; columns: how reliable the clue is (accuracy: 77%, 93%, 99.9%).

Size of the clue	77% reliable	93% reliable	99.9% reliable
0.05	0.035	0.027	0.025
0.2	0.138	0.106	0.100
0.8	0.560	0.422	0.400

In plain words. The flip size is proportional to the clue's *size*: shrink the clue sixteenfold and the flip size shrinks sixteenfold. Making the clue more reliable does not make it safer; if anything, a nearly perfectly reliable clue is slightly *easier* to flip, because with one clue the flip size is just the distance to the decision boundary, and a very reliable clue puts almost every point about half the class gap from it, while a less reliable clue leaves some points farther away. A tiny clue that is 99.9% reliable can be flipped by a nudge of 0.025.

8 Question 5: can we fix it?

Remove the weak clues. Model B, which never sees them, is much harder to fool at large nudge sizes (Table 5): 0.211 flipped at $\epsilon = 0.3$ against 0.536 for A. The price is 3.8 points of accuracy (99.4% $\rightarrow$ 95.6%). Below $\epsilon = 0.10$ there is no visible gain: the curves of the full model and the reliable-clue-only model overlap there (Figures 9 and 8). This is a trade, not a free fix: the weak clues really did carry information.

Table 5: The three models side by side (mean ± s.d. over 10 runs). "Flipped" is the share of correctly classified test points the attack turns into errors, when the attacker may change every input the model has.

Model	Accuracy	Flipped, $\epsilon = 0.10$	Flipped, $\epsilon = 0.30$
A (all six inputs)	0.994 ± 0.003	0.048 ± 0.007	0.536 ± 0.020
B (reliable clue only)	0.956 ± 0.005	0.043 ± 0.007	0.211 ± 0.014
C (weak slots frozen at zero)	0.956 ± 0.005	0.043 ± 0.007	0.211 ± 0.014

Freezing the weak clues at zero (Model C). A cheaper idea than removing the inputs is to keep the slots but always feed zeros. The slots are then never trained, and one might worry that an attacker could still use their leftover random weights, by nudging those "zero" inputs away from zero. In practice not: with ordinary initialisation, nudging those slots flips 0% of points (in all 10 runs), and Model C behaves exactly like Model B (Table 5). But this safety depends on the *size* of the leftover weights (Figure 16). Think of the answer as a weighted sum again. Nudging each frozen slot by ϵ can move the sum by at most ϵ times the total size of those weights. While that is smaller than the network's confidence, nothing flips. If we multiply the leftover weights by 3 or 10 it stays at ≈ 0 (0.000 and 0.001 ± 0.001), but at 100× the weights are so large that the same small nudge on the "zero" inputs overturns the answer (0.92 ± 0.12 flipped).



Figure 16: Model C: share flipped when the attacker nudges only the frozen weak slots, as their leftover weights are multiplied by 1, 3, 10 and 100 (10 runs).

Let the model remove them itself. Instead of deciding in advance which clue is reliable, we can penalise the model for using many inputs. Penalties that switch whole inputs off (the L_1 penalty, group Lasso, elastic net) behave like Model B without being told which clue to keep (Table 6).

Table 6: Model A trained with each recipe, attacked on the five weak clues only, at $\epsilon = 0.3$ (mean $\pm$ s.d. over 10 runs). Baseline: ordinary training. Each trained-penalty row is a setting with accuracy close to 0.97; the last row is ordinary shrinkage at its strongest setting that still works.

Training recipe	Accuracy	Flipped
baseline	0.994 ± 0.003	0.260 ± 0.014
L_1 penalty	0.969 ± 0.005	0.023 ± 0.008
group Lasso	0.975 ± 0.005	0.038 ± 0.012
elastic net	0.979 ± 0.004	0.050 ± 0.008
simply deleting 4 weak inputs afterwards	0.970 ± 0.005	0.052 ± 0.007
ordinary shrinkage (L_2), strength 0.2	0.993 ± 0.003	0.213 ± 0.013

Ordinary shrinkage (L_2, the standard "weight decay") cannot reach that regime: with moderate strength it costs no accuracy and helps a little; at strength 0.4 it collapses to chance, with nothing in between.

Figure 17 shows every setting we tried at once. The selective penalties and deleting inputs trace a smooth curve from "accurate but easy to fool" to "a bit less accurate but hard to fool". Ordinary shrinkage (L_2, red crosses) sits on the accurate end of the same curve and then falls off a cliff. Figure 18 shows the same trade-off for the L_1 sweep, measured by flip size against the best possible point.

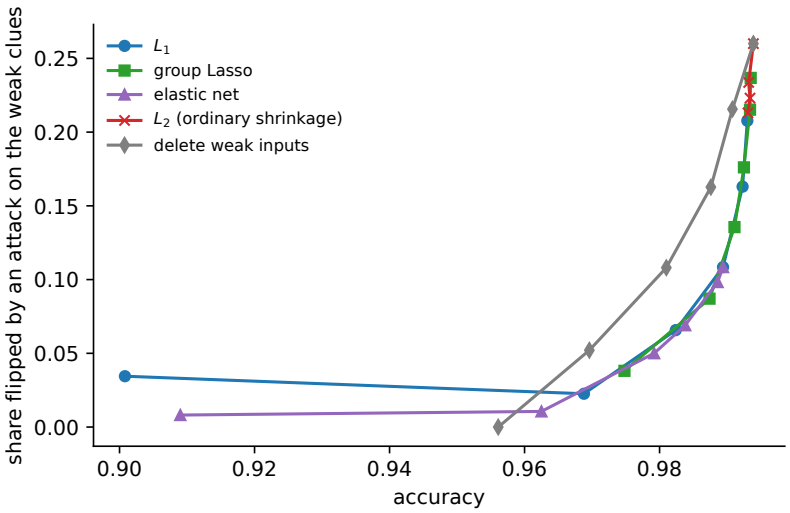


Figure 17: Accuracy against the share flipped by an attack on the weak clues, for every setting of each method (mean over 10 runs). Lower and further right is better. The point at the far left of the L_1 line (strongest penalty) has runs that over-shrink the reliable clue.

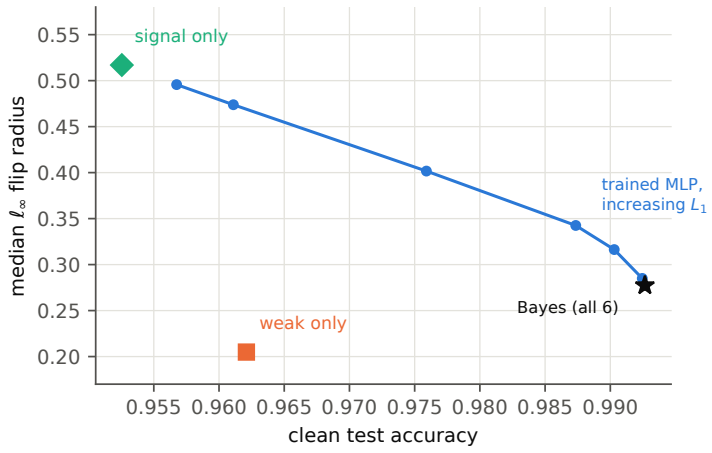


Figure 18: Accuracy against robustness. As the L_1 penalty grows (blue), the model moves from the best possible point (star) toward the reliable-clue-only model (green). The weak-only model (orange) is more accurate than reliable-only but far less robust.

The cost of concentrating. Putting all the weight on the reliable clue has a downside. With the L_1 penalty the weak-clue attack falls from 0.260 to 0.023, but an attack on the reliable clue alone rises from 0.037 to 0.173 (and Model B, which has only that clue, sits at 0.211). With the redundancy gone, the one clue left is a bigger single point of failure. The net effect is still a large win, because the weak-clue loss is so much bigger, but the trade-off exists whether the removal is done by us or by the penalty.

Choosing the penalty strength. A very light L_1 penalty can look like it does nothing if it is compared against the wrong baseline: the weight on the reliable clue relative to the weak ones is 2.4 at strength 0.02 against 2.0 for ordinary training, an unremarkable change. Sweeping the strength shows the ratio climbing from 2.4 to 18 (at 0.4) while accuracy slips only from 99.3% to 96.9% (Figure 19). Pushed too far (0.8), some runs over-shrink even the reliable clue and fall apart (mean accuracy 0.90 ± 0.15). The practical lesson is to compare against a true no-penalty baseline, and to sweep.

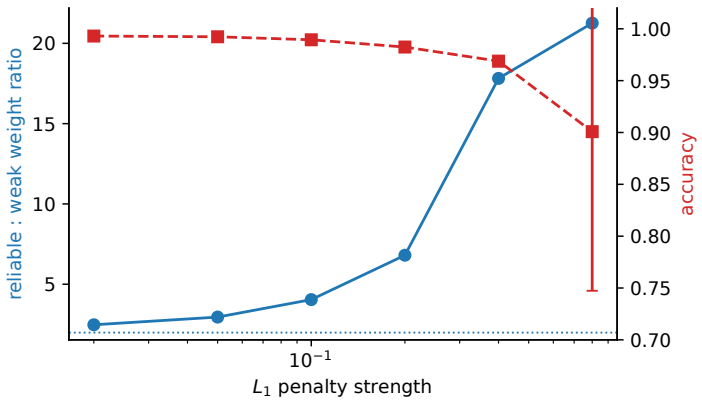


Figure 19: Effect of the L_1 penalty strength (10 runs per point). Blue: ratio of the weight on the reliable clue to the weight on the weak clues (median over runs; dotted line: ordinary training). Red: accuracy, mean $\pm$ s.d. The ratio rises from about 2 to about 18 while accuracy slips mildly; at the strongest value some runs over-shrink the reliable clue itself, which is why the error bar is so wide.

Not every kind of sparsity helps. A different standard penalty discourages *hidden activity* (how many internal units fire) instead of how large the input weights are. It does what it is meant to: the average activity falls from 0.42 to 0.07 and the fraction of silent units rises from 38% to 93%. But it has no handle on *which input* to keep: the reliable-to-weak weight ratio stays flat ($1.97 \to 1.81$), and the weak-clue attack is as effective as ever (0.260 against 0.255–0.276 at every strength up to 2.0). Pushed harder, it fails abruptly instead of gradually: each run either stays fully accurate or becomes a dead network at chance (Figure 20); no run was ever in between. The number of dead runs rises with strength (0 of 10 up to 2.0, 2 at 3.0, 7 at 4.0, 9 at 6.0, all 10 at 8.0), so the *average* accuracy appears to decline smoothly although no single network ever does. The mechanism is simple: the penalty pushes each hidden unit's input negative everywhere, a silenced unit passes no gradient, and so it cannot recover; once every unit is silent the network is dead. A single run would therefore either miss the collapse or mistake it for typical. (We tested one such penalty on this one network; we do not claim it holds for other forms of activation sparsity.)

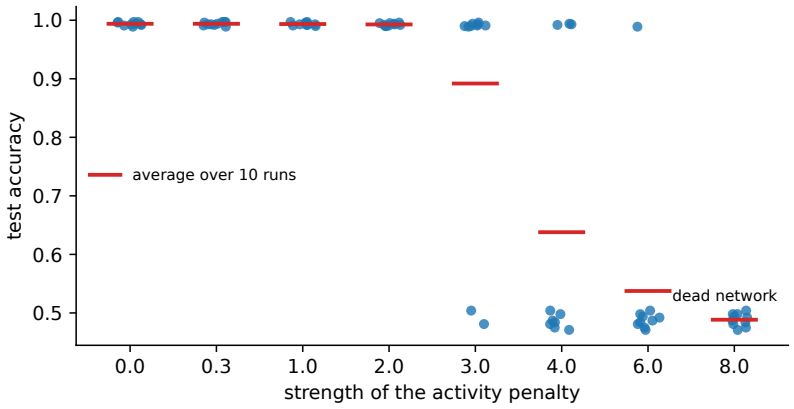


Figure 20: Each dot is one training run (10 per strength). Accuracy is all-or-nothing: functional runs stay near 99%, dead networks sit at chance. The red bar, the average, falls smoothly only because the share of dead runs grows.

Table 7 collects the fixes in one place.

In plain words. Every fix that works does the same thing: it reduces the sum of weights by giving up some of the weak clues' information, so robustness is paid for in accuracy. Penalising the *input weights* lets the model drop whole clues; shrinking everything a little, or penalising hidden *activity*, does not, and the latter breaks networks all at once.

Table 7: Practical summary of the fixes (numbers from this section; attack on the five weak clues at $\epsilon = 0.3$, baseline 0.260 at 0.994 accuracy).

Fix	Effect on the weak-clue attack	Accuracy cost	Failure mode
L_1, group Lasso, elastic net (switch whole inputs off)	large: 0.26 → 0.02–0.05	1.5–3 points	too strong over-shrinks the reliable clue; concentrates the model on one clue (single point of failure)
Delete weak inputs after training	large but smaller: 0.052 at 0.970 accuracy	about 2.4 points	worse trade than L_1 at the same accuracy
Remove weak inputs by design (Model B)	full-attack share 0.536 → 0.211	3.8 points	needs to know in advance which clue is reliable
L_2 (ordinary weight decay)	small: 0.260 → 0.213	none, until it collapses	abrupt collapse to chance at strength 0.4
Activity penalty	none: 0.255–0.276	none, until it collapses	each run is fully accurate or dead; no input selection

9 Question 6: when does this happen at all?

We retrained the network over a grid of how noisy the reliable clue is, how strong the weak clues are, and how much weight decay is used (Figure 21).

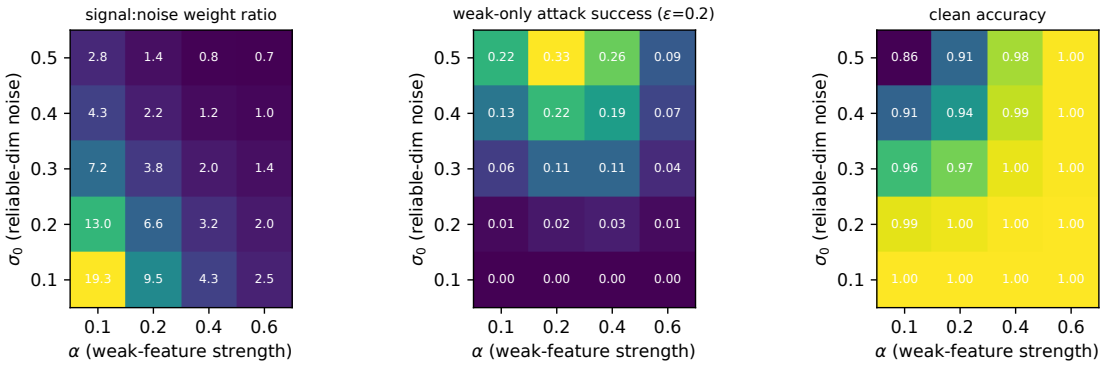


Figure 21: Weight decay 10^{-2}. Left: reliable-to-weak weight ratio. Middle: share flipped by an attack on the weak clues alone ($\epsilon = 0.2$). Right: clean accuracy. Rows: noise of the reliable clue (lowest at the bottom); columns: strength of the weak clues.

- If the reliable clue is nearly perfect (lowest noise, bottom row), the network ignores the weak clues and the effect disappears: at the weight decay shown the weak-clue attack flips at most 0.1% of points in that row, and at most about 1% at any weight decay on the grid. The network has no pressure to use clues it does not need, even though they carry information.
- The effect grows as the reliable clue gets noisier, and peaks for moderately strong weak clues (up to 0.37 flipped).
- Weight decay is *not* what creates the effect. Turning it up makes the network rely *less* on the weak clues (for example the flipped share falls $0.25 \to 0.24 \to 0.22 \to 0.11$ as weight decay rises from none to the strongest value, at one setting of the grid).

10 Does any of this happen on real images?

Only partly, and the honest summary is more interesting than a clean confirmation. We look at three image tasks.

10.1 Circles and squares

We built a small picture version of the toy world. Each 28×28 image shows a filled circle (label 0) or square (label 1) on a gray background; pixel values lie in $[0, 1]$. The shape *is* the label (a network sees it with 99.9% accuracy alone). The background gray level is only correlated with the label (73.8% accuracy alone). Noise is added to every pixel, so the shape is hard to see by eye (Figures 22, 23), yet a small convolutional network reads it at 99.6%. We split each image into two regions: the *shape region*, a central box that always contains the shape (the shape shifts by up to two pixels), and the *border*, the frame of pixels outside that box, which never touches the shape. (One-step attack, 10 runs.)

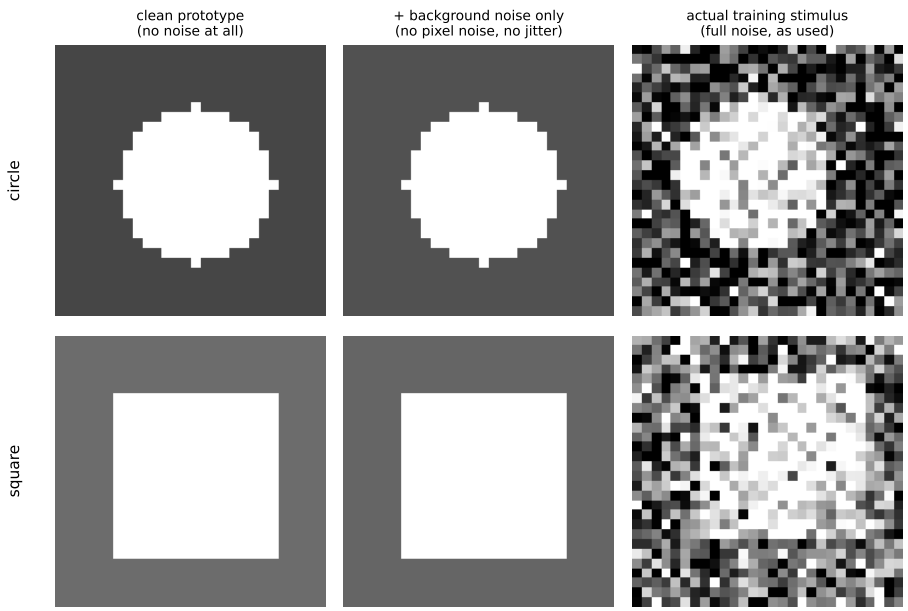


Figure 22: Left: noise-free prototypes of each class (the background shade difference is visible). Middle: one sample of background-level noise. Right: an actual training image.

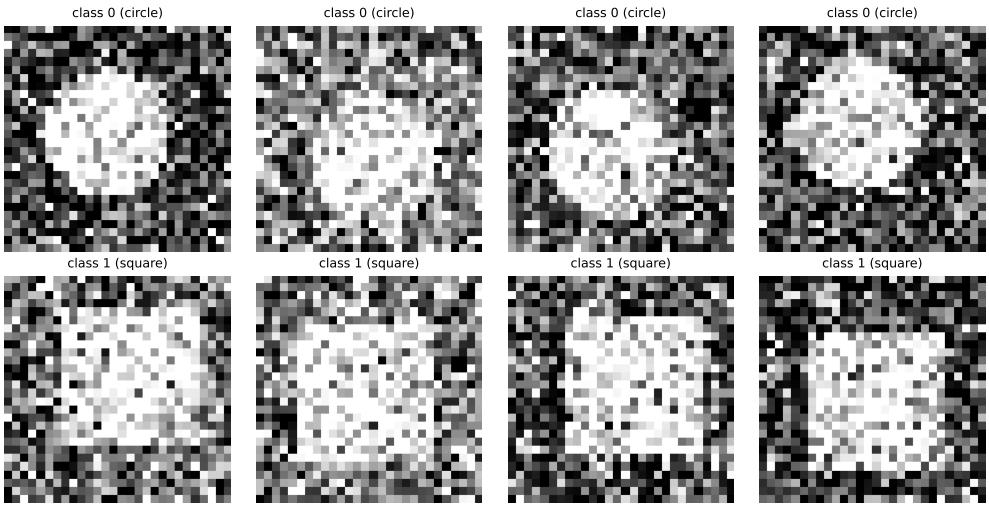


Figure 23: Training examples. The background level varies within and across classes.

Surprise. In the toy task, a near-perfect reliable clue made the network ignore the weak clues, so attacking the weak clues did nothing (Section 9). Here the shape is nearly perfect, yet the network still picks up the background: an attacker who may change *only the border pixels*, never touching the shape, flips 46%–71% of answers at nudge sizes 0.2–0.4. We do not have a tested explanation, but a plausible one is that the background level is shared by every border pixel, so pooling over many pixels gives the network a cheap, low-noise estimate of it, much easier to read than a noisy shape. (In Table 8, "background border" is the border region just defined.)

Table 8: Share of image answers flipped (one-step attack), by where the attacker may change pixels (mean ± s.d. over 10 runs).

nudge size	all pixels	shape region	background border
0.10	0.942 ± 0.037	0.587 ± 0.055	0.104 ± 0.076
0.20	1.000 ± 0.000	0.996 ± 0.005	0.464 ± 0.073
0.30	1.000 ± 0.000	1.000 ± 0.001	0.668 ± 0.082
0.40	1.000 ± 0.000	0.996 ± 0.013	0.711 ± 0.084

A trap in the comparison. The "shape region" column is *not* the picture version of "reliable clue only". In the toy task, nudging the reliable clue moves one number a little. In an image the shape region contains the shape's own pixels, and a large enough change simply redraws the shape into something else (Figure 24). The shape pixels are the ground truth, not a measurement of it. So the border-only attack is the one that matches the intended meaning of "adversarial": it exploits an incidental clue at a nudge too small to redraw anything.

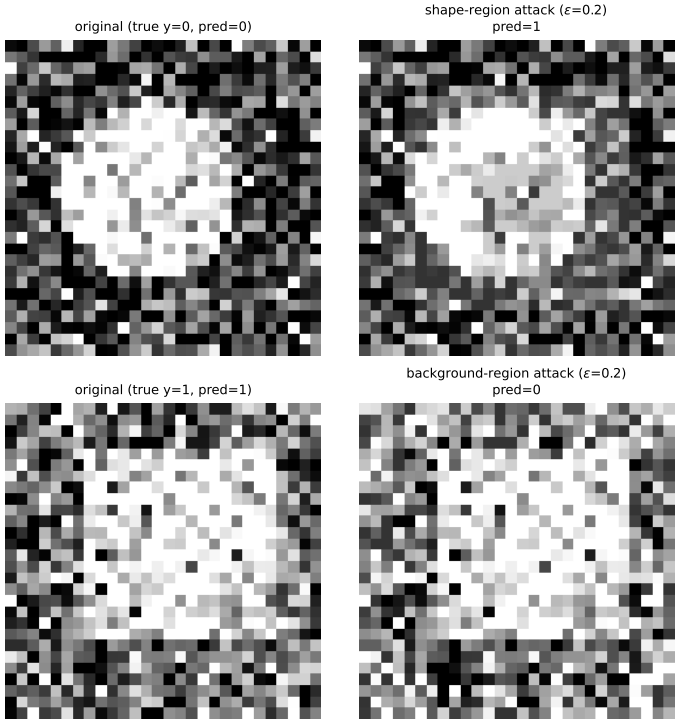


Figure 24: One run, nudge size 0.2. Top: an attack confined to the shape region visibly distorts the shape. Bottom: an attack confined to the border changes only the background, leaves the shape untouched, and still flips the answer.

Genuine separation helps (with a caveat). We built a two-tower network in which one tower sees only the shape region and the other only the rest, so restricting the attack to a region restricts it to one tower's whole input. The border-tower attack is clearly weaker than the single network's border attack (0.312 against 0.668 at nudge size 0.3; 0.454 against 0.711 at 0.4) at the same accuracy (99.7%), 5 runs. The shape-tower attack stays near 1.0, as expected. We cannot claim a clean ablation: the two-tower network has about twice the parameters and narrower connections,

so some of the effect may be capacity, not separation. An iterated attack (PGD-10, 2 restarts, 5 runs) is a little stronger than the one-step attack on this task (border attack at 0.3: 0.654 → 0.752), but the pattern is unchanged.

10.2 MNIST

We scrambled the pixels of handwritten digits with a random rotation of the input space that leaves all distances and all information unchanged, but spreads each clue over more coordinates. The straight-line size of the change needed stays the same (accuracy too), as predicted. The per-pixel flip size, however, falls by only 12–14%, not the eightfold drop of the toy world. Digits in their ordinary pixel form are already nearly as spread out as an input can be, so there is little room left to spread (Figure 25).

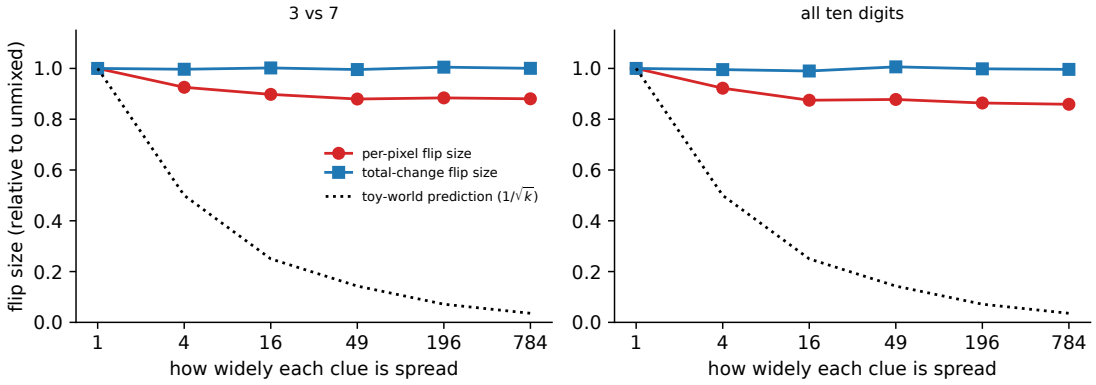


Figure 25: MNIST after mixing pixels within blocks of size k (so each clue is spread over k inputs). Blue: total-change flip size, unchanged as predicted. Red: per-pixel flip size, which drops only slightly. Dotted: what the toy world predicts ($1/\sqrt{k}$). Relative to $k = 1$; 5 runs.

10.3 CIFAR-10

We let a small image network use only the strongest m patterns in the images. More patterns means higher accuracy (44% → 78%) but a 6.8× smaller per-pixel flip size. Here the loss does *not* come from spreading; it comes from the model using finer, smaller-margin patterns that are individually easier to move. (These medians are over correctly classified points, so models with few patterns are measured on an easier subset, which overstates their robustness.) Figure 26 shows the curves.

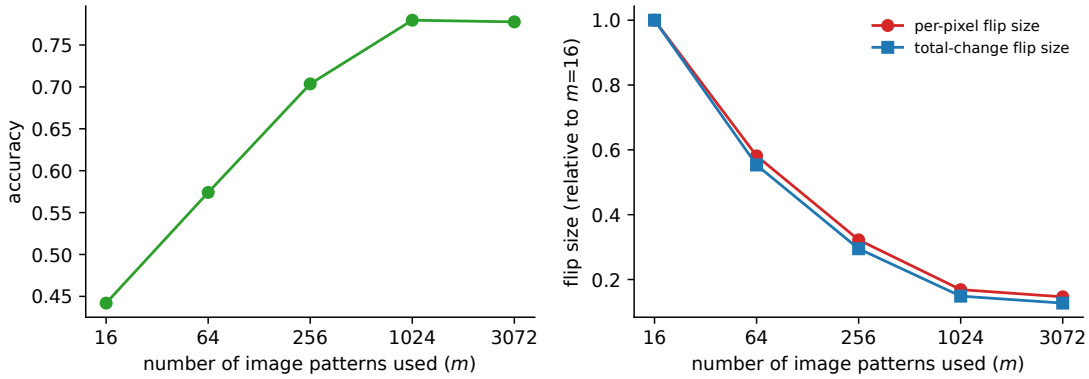


Figure 26: CIFAR-10 with the network allowed to use only the top m image patterns. Left: accuracy rises with m. Right: both flip sizes fall together, by about seven to eight times, so the loss is a shrinking margin, not spreading (5 runs).

In plain words. The adding-up geometry is real on real images too, but natural images are already in the "spread out" regime, so the toy world's eightfold effect is not what drives them. On CIFAR, the cost of using more detail is mostly smaller margins. What we have not shown: that large deep networks on natural data work this way.

11 Discussion: what these analyses say about the cause

Putting the pieces together. In the toy world, vulnerability to small per-input changes comes from three ingredients acting together. First, the model's answer is a *sum* of many contributions, so $\|w\|_1$ is large (Section 4). Second, a per-input budget lets the attacker push all of those contributions in the helpful direction at once. Third, clues that are small compared with the allowed change are easy to move, however reliable they are (Section 7). No single feature is the culprit, and the cause is not a training flaw: the best possible rule has the same weakness (Section 5). Accuracy and vulnerability are linked, because the weak clues that make the model accurate are also the ones that add up to make it fragile, so robustness bought by removing them costs accuracy (Section 8).

How far this carries to real images. On real images the picture is partly confirmed. The geometry is the same (Section 10), but pixel data are already in the "spread out" regime, so spreading further changes little, and on CIFAR-10 the cost of using more detail shows up mostly as smaller margins. The toy world is therefore best read as a way to see the mechanism, not as a measurement of how much of a real network's vulnerability it explains.

Why might human vision be reliable? We have no experiment on people, so this is a hypothesis, not a result. In our language, a classifier is easy to fool when it relies on many tiny clues, each small compared with a small change. One idea is that human recognition leans more on a few large-scale properties, such as overall shape, which are large compared with a faint pixel-level change. This echoes work by Geirhos et al. [6], who found that standard image networks lean on texture more than people do, and that making them rely more on shape improves robustness. Fine texture is the kind of clue that is spread over many pixels and small in each. In our circles-and-

squares task the shape is a large-scale clue and the background level a spread-out one, and the network used both.

One-pixel attacks. Su et al. [7] showed that changing a *single* pixel, by a large amount, can sometimes fool a network. This is a different budget: very few inputs but unlimited size, instead of every input but tiny size. In the weighted-sum picture, changing one input by δ moves the total by $\delta|w_i|$, so a single input can flip the answer only if its weight is large compared with the confidence. Spreading evidence over k inputs makes each weight smaller (by $1/\sqrt{k}$), so the picture predicts that spreading evidence makes small-change attacks easier but single-input attacks harder. We have not tested this prediction.

Geometric distortions and invariance. Our attacker edits values directly, and the toy world has no geometry. Natural distortions such as small rotations and shifts change many inputs together in a structured way, and Engstrom et al. [8] showed that such transformations alone can fool networks. Robustness to them needs a different property: the model's internal representation should stay (or change predictably) under the transformation, which is the idea of invariance and equivariance. We leave the following open and untested here: if a representation is equivariant to a transformation, does an attack made at one viewpoint transfer predictably to another? By the chain rule the gradient, and so an attack built from it, would transform predictably too. The toy world could be extended with a transformation parameter to study this.

In short. A model that adds up many small, individually reasonable pieces of evidence is accurate, and for the same reason a coordinated small change to all of them can overturn its answer. That is our candidate for the main cause of adversarial vulnerability; whether it is the *only* cause in large networks on natural images is not settled by this paper.

12 Takeaways and limits

1. Spreading the same predictive evidence over more inputs makes a model easy to fool when each input may be nudged a little (ℓ_∞): the per-input change needed shrinks like $1/\sqrt{k}$, while the total change needed and the accuracy do not move.
2. How *reliable* a clue is does not make it safe; how *small* it is relative to the nudge does. A 99.9%-reliable clue can be flipped by a tiny change.
3. Weak clues are not useless, and no single weak clue is the culprit; the vulnerability is in the sum.
4. In the toy task of this paper it is a property of the problem and the attack rule, not a flaw in training: the best possible classifier is just as vulnerable.
5. Fixes work by throwing away information, and cost accuracy; switching whole inputs off works better than shrinking everything, and concentrating on one clue creates a new single point of failure.
6. Sparsity of the *weights* helps; sparsity of the *activity* does not, and breaks networks all at once.
7. Always judge the nudge against the size of the clue, and beware near-perfect clues, which can hide the effect entirely.

Limits. One toy task with independent clues, a single small network, and a one-step attack in the main comparisons. The real-data checks support the geometry but not the toy world's size of effect. We did not test correlated clues, deep networks, or adversarial training.

A Experimental details

Toy task. Given the label y, each input is an independent bell-curve variable with mean $(y - \frac{1}{2})\Delta\mu_i$ and standard deviation σ_i, no clipping. Reliable clue: $\Delta\mu = 1.0$, $\sigma = 0.3$. Weak clues: $\Delta\mu = 0.4$, $\sigma = 0.25$. The best possible rule is a weighted sum with weights $\Delta\mu_i/\sigma_i^2$ (11.1 and 6.4) and is exact. Single-clue accuracy is $\Phi(\Delta\mu/2\sigma)$.

Networks and training. One hidden layer of 8 ReLU units, two outputs; Adam (learning rate 0.02), 500 full-batch steps, weight decay 10^{-2} unless varied; 4,000 training and 1,000 test points (2,000 test points for the flip-size experiments). Each of the 10 runs reseeds the data and the initialisation. Quantities are reported as mean ± one sample standard deviation across runs. Model C's leftover weights are scaled before training (they receive no gradient, so they stay at that scale).

Attacks. "Flipped" is the share of correctly classified test points whose predicted label changes after the one-step attack (nudge ϵ times the sign of the loss gradient, optionally masked to a subset of inputs). "Flip size" is the smallest nudge that flips a given point: computed exactly for the best possible rule ($|\text{LLR}|/\|w\|_1$ per input, or $|\text{LLR}|/\|w\|_2$ for straight-line distance) and by per-point bisection over a 20-step attack for trained networks (14 bisection steps). The tables report the median flip size over points, then average over runs.

Spread experiment. Total discriminability $d' = 1/0.3$ (accuracy 95%) split equally over $k \in \{1, 2, 4, \ldots, 64\}$ inputs, each with noise 0.25; a network is trained at each k (10 runs). **Size-versus-reliability experiment:** one input with class gap $\Delta\mu \in \{0.05, \ldots, 0.8\}$ and reliability $d' \in \{1.5, 3, 6\}$ (noise $\Delta\mu/d'$), with a fixed standardising layer so tiny inputs are not handicapped by the optimiser; attacks act on the raw input.

Regularisers. L_1 and group Lasso penalties act on the first-layer weights with no other weight decay (group Lasso: one norm per input column); elastic net uses $L_1 = 0.1$ with the L_2 strength varied; pruning zeros the smallest-norm weak columns after ordinary training with no retraining. The activity penalty adds λ times the mean hidden activation. A run counts as a dead network if its test accuracy is below 0.6.

Phase diagram. Reliable-clue noise $\in \{0.1, \ldots, 0.5\}$, weak-clue gap $\in \{0.1, 0.2, 0.4, 0.6\}$, weight decay $\in \{0, 10^{-3}, 10^{-2}, 10^{-1}\}$; 5 runs per cell; attack size 0.2.

Images. Circles and squares: 28×28 grayscale, shape position jittered by ± 2 pixels, background level shifted by class with overlapping noise, plus per-pixel noise; a two-layer convolutional network; 10 runs (5 for the two-tower network and the iterated-attack check). Figure 1 uses the unmixed MNIST network below, with the smallest of the nudge sizes 0.02–0.08 at which an iterated attack flips the answer. MNIST: a 784–256–10 network trained with momentum SGD on 10,000 images, 1,000 test images, 5 runs, inputs rotated within random blocks of size $k \in \{1, 4, 16, 49, 196, 784\}$. CIFAR-10: a five-layer convolutional network on the top-m principal

components ($m \in \{16, 64, 256, 1024, 3072\}$) of the images; 20,000 training and 500 test images, 5 runs; the attacker perturbs raw pixels with no clipping. Code for all experiments, with fixed seeds and the settings above, is at `https://github.com/alikayyam/robust`.

B Glossary

Adversarial example An input changed slightly, on purpose, so that a model gets it wrong.

Clue / feature A number the model reads. *Reliable* if it alone predicts the label well; *weak* if not; *small* if its two class averages are close compared with the allowed nudge.

Nudge size ϵ (ℓ_∞) The most the attacker may change each input; a "per-input" budget.

Total change (ℓ_2) The straight-line size of the whole change taken together; a different budget, under which spreading evidence does not make a model easier to fool.

Flip size The smallest nudge size at which a given answer flips.

Best possible (Bayes-optimal) classifier The rule with the highest achievable accuracy given how the data were generated; here, a weighted sum of the clues, answered by its sign.

Confidence How far the weighted total is from zero; the farther, the surer the answer.

Scramble Shuffle one clue's values among the test examples so it carries no information about the label.

L_1, group Lasso, L_2 Training penalties on the model's weights. L_1 and group Lasso push whole inputs to zero; L_2 ordinary weight decay shrinks everything.

References

[1] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus. *Intriguing properties of neural networks*. ICLR, 2014.

[2] I. Goodfellow, J. Shlens, C. Szegedy. *Explaining and Harnessing Adversarial Examples*. ICLR, 2015.

[3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. ICLR, 2018.

[4] A. Athalye, N. Carlini, D. Wagner. *Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples*. ICML, 2018.

[5] D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, A. Madry. *Robustness May Be at Odds with Accuracy*. ICLR, 2019.

[6] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, W. Brendel. *ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness*. ICLR, 2019.

[7] J. Su, D. V. Vargas, K. Sakurai. *One pixel attack for fooling deep neural networks*. IEEE Transactions on Evolutionary Computation, 2019.

[8] L. Engstrom, B. Tran, D. Tsipras, L. Schmidt, A. Madry. *Exploring the Landscape of Spatial Robustness*. ICML, 2019.

[9] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, A. Madry. *Adversarial Examples Are Not Bugs, They Are Features*. NeurIPS, 2019.